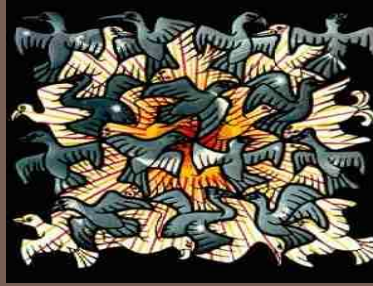


ARQUITETURA E PADRÕES DE SOFTWARE

Prof. Luciano Vale, Msc
lucianovale@unifor.br

Material do Prof. Eduardo Bezerra



PADRÕES GANG OF *FOUR*

PARTE I

Padrões GoF – parte I

3

- ☐ Singleton
- ☐ Composite
- ☐ Façade
- ☐ Adapter
- ☐ Template Method
- ☐ Strategy
- ☐ Iterator

☐	Padrões de Criação
☐	Padrões Estruturais
☐	Padrões de Comportamento

Singleton

Singleton

5

- **Intenção:** Garantir que uma classe tenha uma única instância, e prover um ponto de acesso global a ela.
 - ▣ GoF: *“Ensure a class only has one instance and provide a global point of access to it.”*
- **Aplicabilidade:** usar quando exatamente uma instância da classe é necessária.
- **Exemplos:**
 - ▣ Único acesso ao banco de dados
 - ▣ Único acesso ao arquivo de log

Singleton

6

□ Estrutura

Singleton
- <u>instance : Singleton</u> - data : State
+ <u>getInstance() : Singleton</u> + getData() : State + setData(newData : State) : void

Singleton

7

- Exemplo de código

```
public class Singleton {  
    private static Singleton instance = null;  
  
    public synchronized static Singleton getInstance() {  
        if (instance == null) {  
            instance = new Singleton();  
        }  
        return instance;  
    }  
  
    private Singleton() {  
  
    }  
}
```

Singleton

8

```
public class Singleton {  
    public final static Singleton INSTANCE = new Singleton();  
  
    private Singleton() {  
    }  
}
```

```
public class ClienteDoSingleton {  
    ...  
    obj = Singleton.INSTANCE;  
    ...  
}
```

Composite

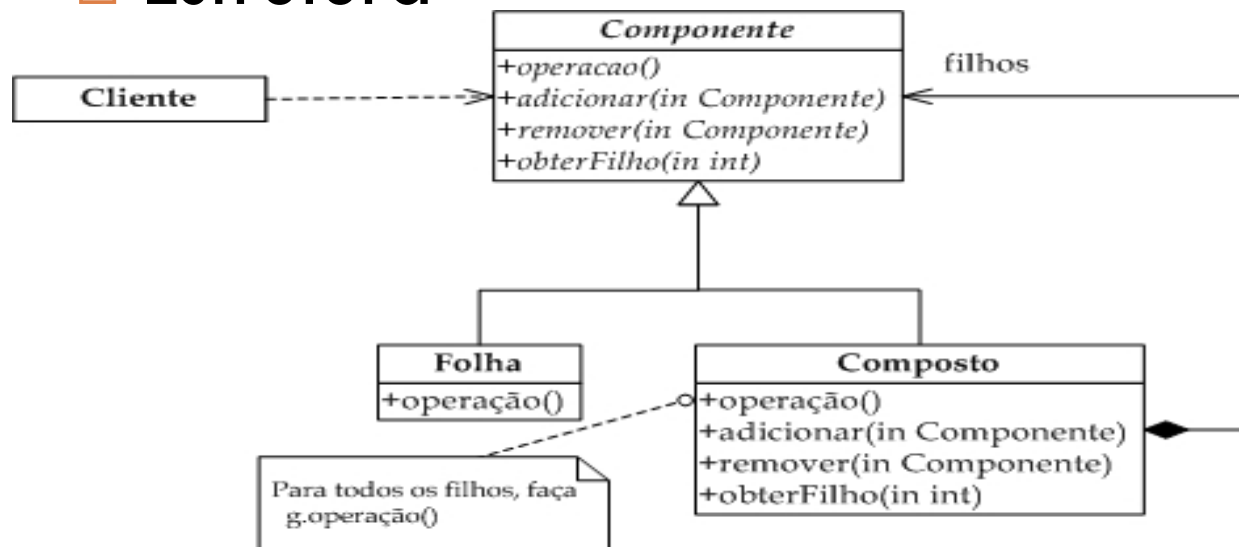
Composite

10

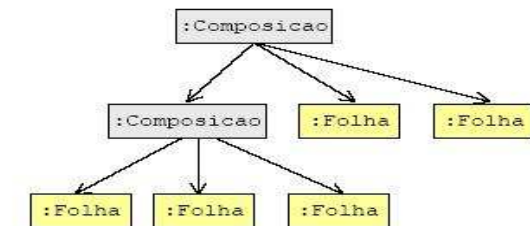
- Intenção: Compor objetos em uma estrutura de árvore para representar **hierarquias todo-parte recursivas**. Permitir ao **cliente** tratar o “todo” (i.e., a composição) e suas “partes” de maneira **indistinta**.
- ▣ GoF Book: *“Compose objects into tree structures to represent part-whole hierarchies. Lets clients treat individual objects and compositions of objects uniformly.”*

Composite

□ Estrutura



O padrão GoF Composite representa uma hierarquia de composição recursiva: um composto possui diversos componentes, que podem ser ou folhas ou outros compostos.



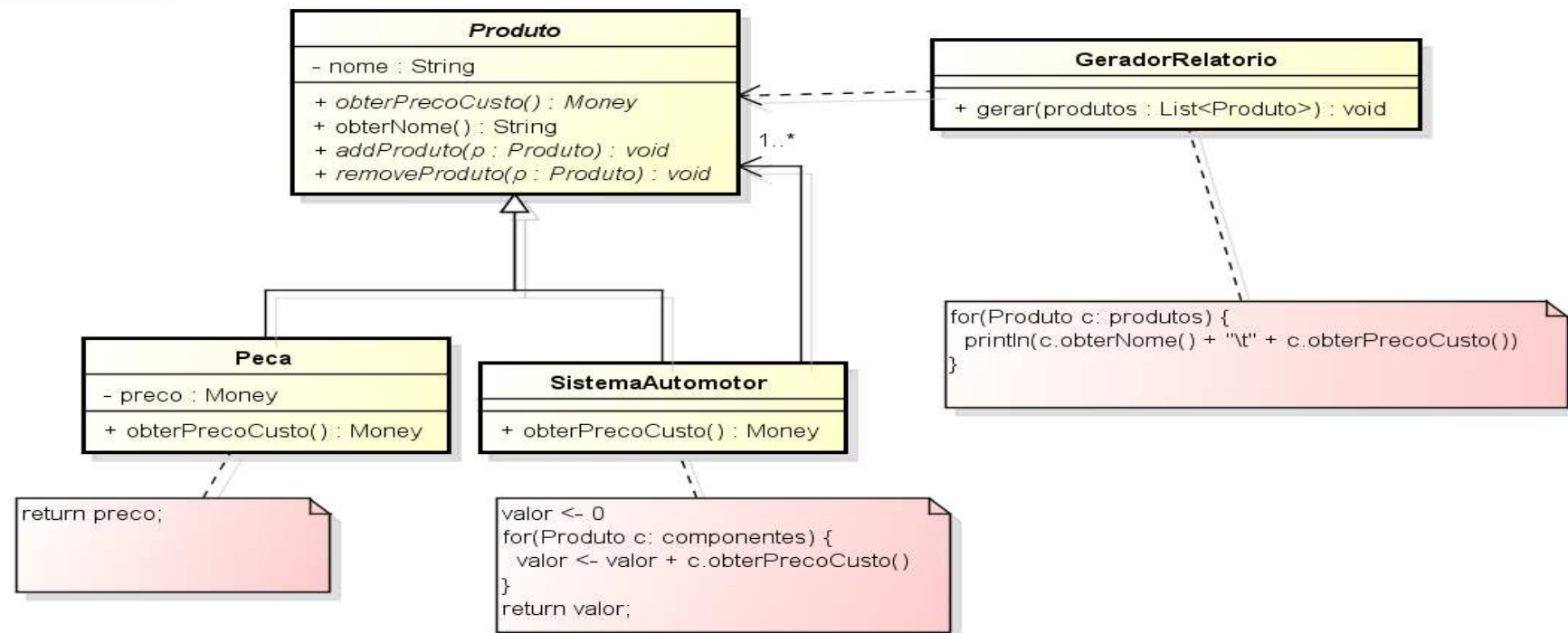
Composite – exemplo 1

12

- Produtos de uma fábrica podem ser **sistemas automotores** ou **peças**.
- Sistemas automotores são compostos de peças simples e de outros sistemas automotores.
- Produtos possuem nome e preço.
- É preciso gerar um relatório de produtos, no qual aparece nome e preço de custo de cada produto, independentemente de ser um sistema automotor ou uma peça simples.

Composite – exemplo 2

13



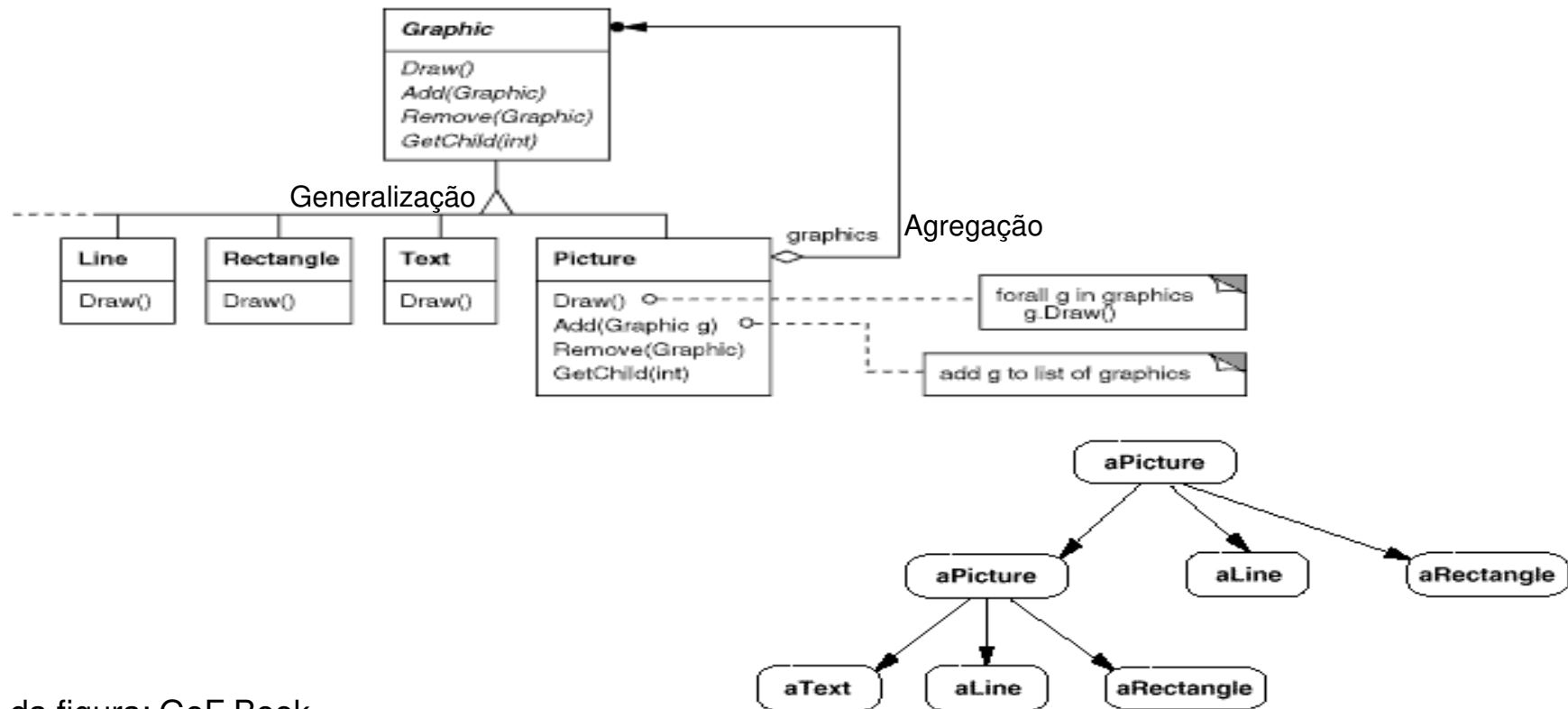
Composite – exemplo 2

14

- Em aplicações de manipulação de objetos gráficos, usuários manipulam uma série de diferentes figuras (linhas, retângulos, texto, etc.).
- Figuras podem em muitos casos ser também agrupadas em figuras mais complexas.
- Parte desse sistema devem tratar de forma distinta cada um dos objetos, mas há também partes em que eles devem ser tratados indistintamente.

Composite – exemplo 2 (cont.)

15

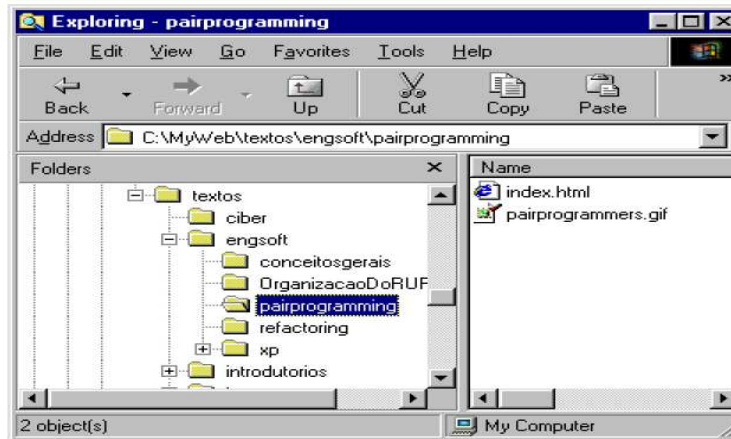


Fonte da figura: GoF Book

Composite – exemplo 3

16

- Em um sistema de arquivos:
 - diretórios são formados de outros diretórios e de arquivos.
 - há uma interface comum para todo e partes. Qual?



Façade

Façade

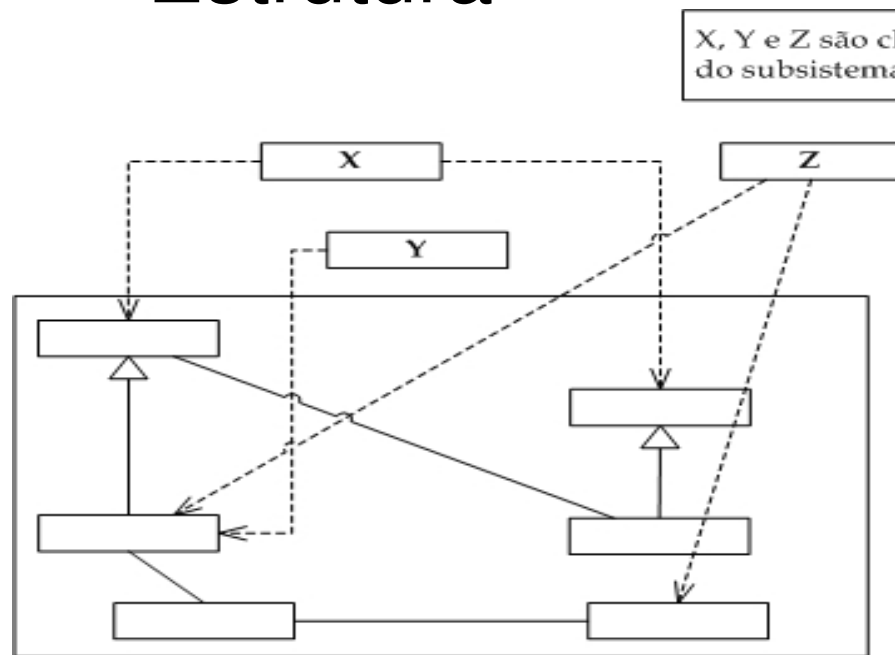
18

- Intenção: Oferecer uma interface única de nível mais elevado e simplificada para um conjunto de interfaces de um subsistema.
 - ▣ *“Provide a unified interface to a set of interfaces in a subsystem. Defines a high level interface that makes the subsystem easier to use.”*

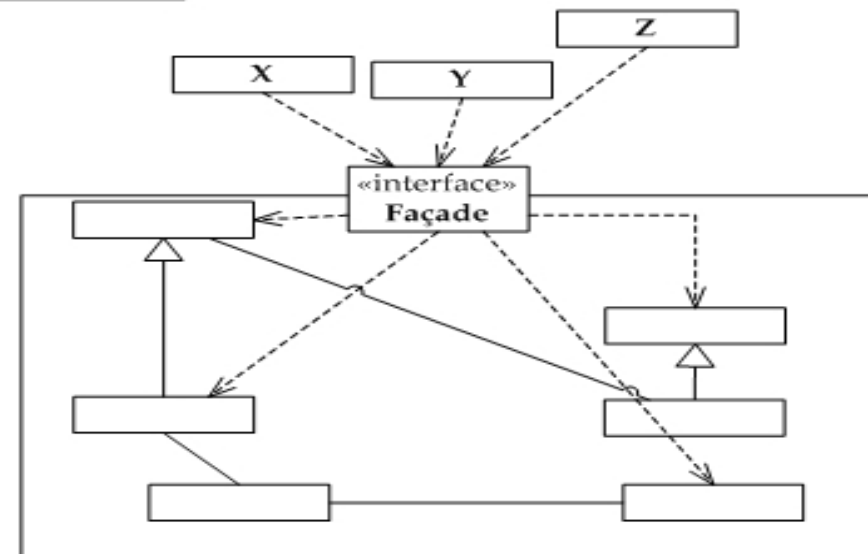
Façade

19

- Estrutura



Solução sem o uso de um objeto Façade



Solução com o uso de um objeto Façade

Façade - exemplo

20

```
class Aplicação {  
    ...  
    Facade f;  
    // Obtem instancia f  
    f.registrar("Zé", 123);  
    f.comprar(223, 123);  
    f.comprar(342, 123);  
    f.fecharCompra(123);  
    ...  
}
```

```
public class Facade {  
    BancoDeDados banco = Sistema.obterBanco();  
    public void registrar(String nome, int id) {  
        Cliente c = Cliente.create(nome, id);  
        Carrinho c = Carrinho.create();  
        c.adicionarCarrinho();  
    }  
    public void comprar(int prodID, int clienteID) {  
        Cliente c = banco.selectCliente(clienteID);  
        Produto p = banco.selectProduto(prodID);  
        c.getCarrinho().adicionar(p);  
    }  
    public void fecharCompra(int clienteID) {  
        Cliente c = banco.selectCliente(clienteID);  
        double valor = c.getCarrinho.getTotal();  
        banco.processarPagamento(c, valor);  
    }  
}
```

```
public class Carrinho {  
    static Carrinho create() {...}  
    void adicionar(Produto p) {...}  
    double getTotal() {...}  
}
```

```
public class Produto {  
    static Produto create(String nome,  
                           int id, double preco) {...}  
    double getPreco() {...}  
}
```

```
public class Cliente {  
    static Cliente create(String nome,  
                           int id) {...}  
    void adicionarCarrinho(Carrinho c) {...}  
    Carrinho getCarrinho() {...}  
}
```

```
public class BancoDeDados {  
    Cliente selectCliente(int id) {...}  
    Produto selectProduto(int id) {...}  
    void processarPagamento() {...}  
}
```

Fonte da figura: Argo Navis, curso J930

Adapter

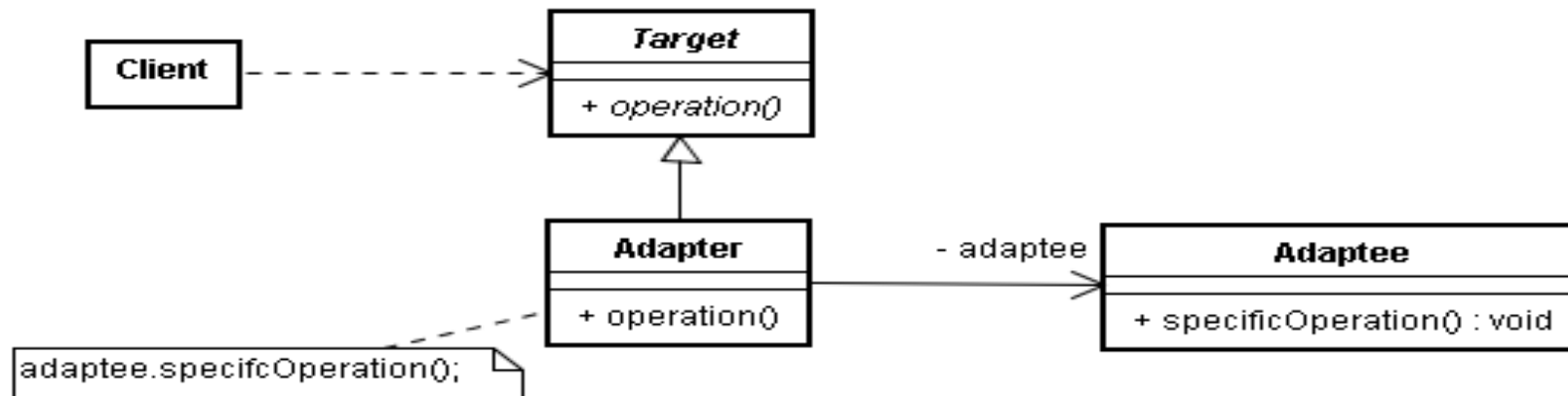
Adapter

22

- **Intenção:** converter a interface de uma classe em outra interface esperada pelos clientes. Permitir que classes com interfaces incompatíveis sejam usadas em conjunto.
- GoF Book: “Convert the interface of a class into another interface clients expect. Lets classes work together that couldn't otherwise because of incompatible interfaces.”

Object Adapter (estrutura)

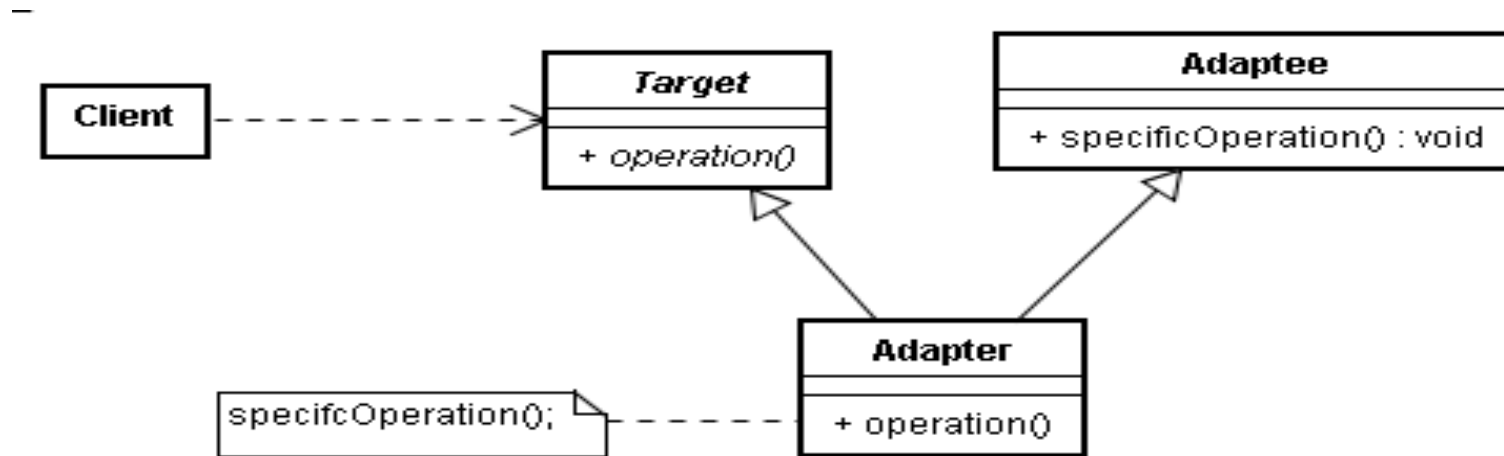
23



Object Adapter: adaptação por composição

Class Adapter (estrutura)

24

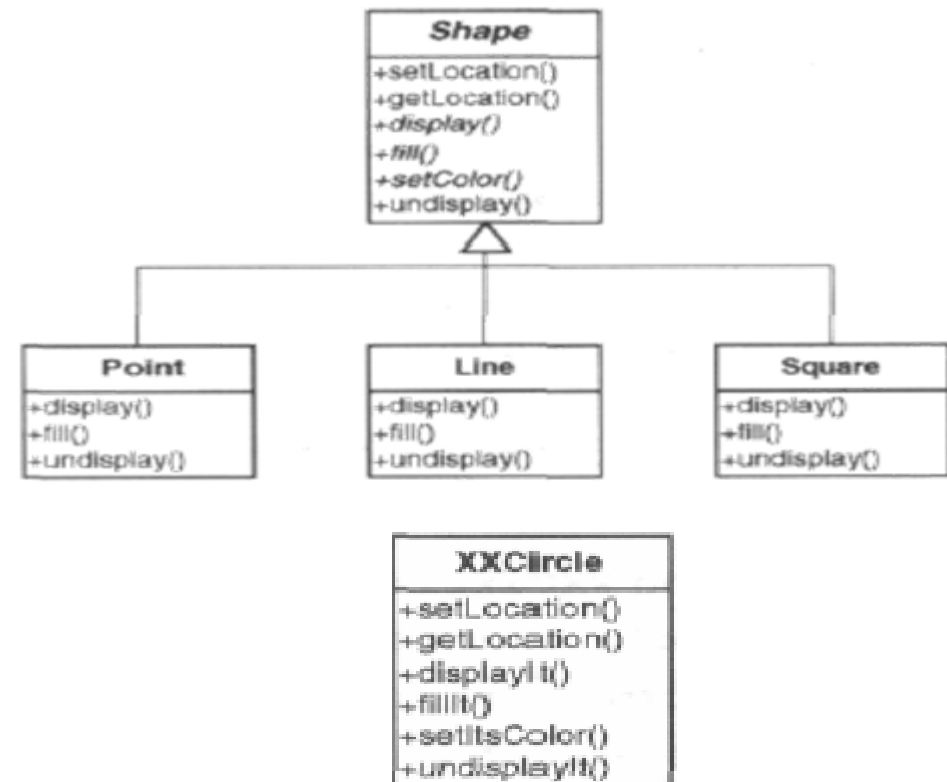


Classe Adapter: adaptação por herança

Object Adapter - exemplo

25

- ❑ Problema: como “encaixar” (adaptar) `XXCircle` à hierarquia de `Shape`, de tal forma que o cliente não seja alterado?
- ❑ Complicadores:
 - ❑ `XXCircle` não pode ser usado diretamente pelo cliente, pois o comportamento polimórfico em `Shape` deve ser preservado.
 - ❑ As listas de parâmetros são diferentes.

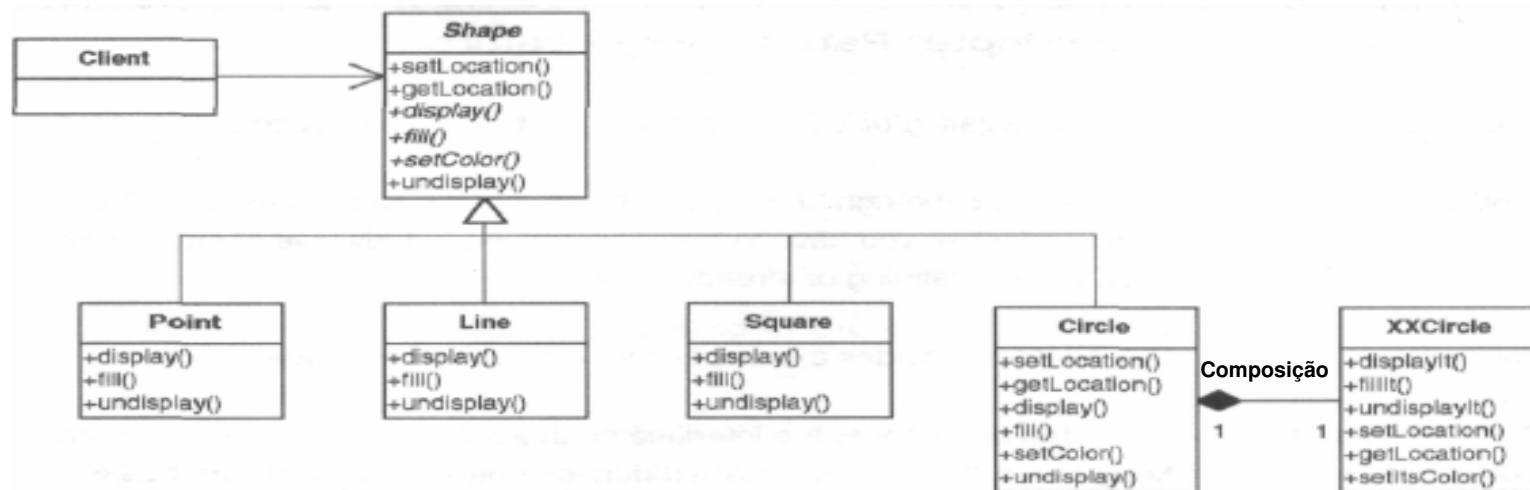


Fonte: Alan Shalloway, James Trott - *Design Patterns Explained*

Object Adapter – exemplo (cont.)

26

- Solução: definir uma classe Circle que deriva de Shape e contém um XXCircle.
- ▣ Essa classe delega mensagens diretamente para XXCircle.



Class Adapter (exemplo de código)

27

```
3 public class Circle extends XXCircle implements Shape {
4     @Override
5     public void display() {
6         this.displayIt();
7     }
8
9     @Override
10    public void fill() {
11        this.fillIt();
12    }
13
14 }
```

```
3 public class XXCircle {
4     public void fillIt() {
5         System.out.println("XXCircle.fillIt()");
6     }
7
8     public void displayIt() {
9         System.out.println("XXCircle.displayIt()");
10    }
11 }
```

```
3 public class Principal {
4
5     public static void main(String[] args) {
6         Shape[] shapes = {new Line(), new Square(), new Circle()};
7         exibirImagens(shapes);
8     }
9
10    private static void exibirImagens(Shape[] shapes) {
11        for (Shape shape : shapes) {
12            shape.display();
13            shape.fill();
14        }
15    }
16
17 }
```

```
3 public interface Shape {
4     public void fill();
5     public void display();
6 }
7
```

Object Adapter (exemplo de código)

28

```
3 public class Circle implements Shape {
4     private XXCircle adaptee = new XXCircle();
5
6     @Override
7     public void display() {
8         adaptee.displayIt();
9     }
10
11    @Override
12    public void fill() {
13        adaptee.fillIt();
14    }
15 }
16 }
```

```
3 public class XXCircle {
4     public void fillIt() {
5         System.out.println("XXCircle.fillIt()");
6     }
7
8     public void displayIt() {
9         System.out.println("XXCircle.displayIt()");
10    }
11 }
```

```
3 public class Principal {
4
5     public static void main(String[] args) {
6         Shape[] shapes = {new Line(), new Square(), new Circle()};
7         exibirImagens(shapes);
8     }
9
10    private static void exibirImagens(Shape[] shapes) {
11        for (Shape shape : shapes) {
12            shape.display();
13            shape.fill();
14        }
15    }
16
17 }
```

```
3 public interface Shape {
4     public void fill();
5     public void display();
6 }
7
```

Template Method

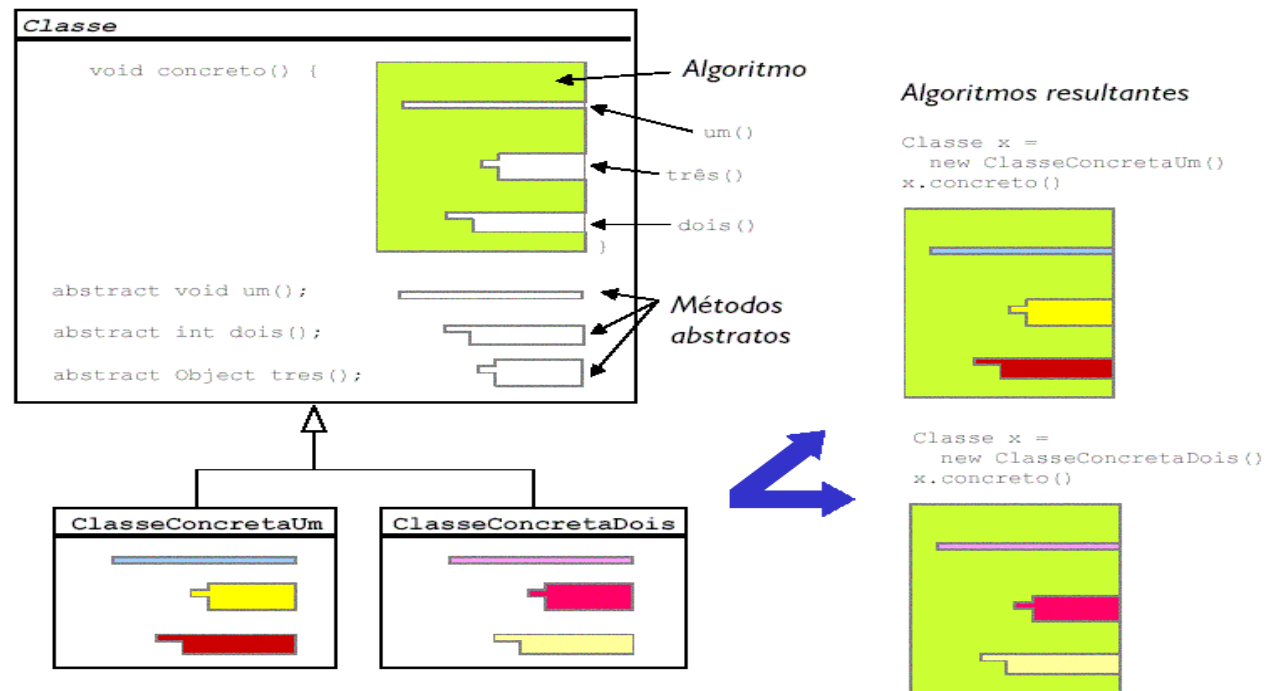
Template Method

30

- Intenção: Definir o **esqueleto de um algoritmo** dentro de uma **operação** em uma classe X, deixando alguns passos para serem preenchidos pelas subclasses de X. Permite que subclasses redefinam certos passos de um algoritmo sem alterar a estrutura desse algoritmo.
- ▣ GoF Book : “Define the skeleton of an algorithm in an operation, deferring some steps to subclasses. Lets subclasses redefine certain steps of an algorithm without changing the algorithm's structure.”

Template Method

31

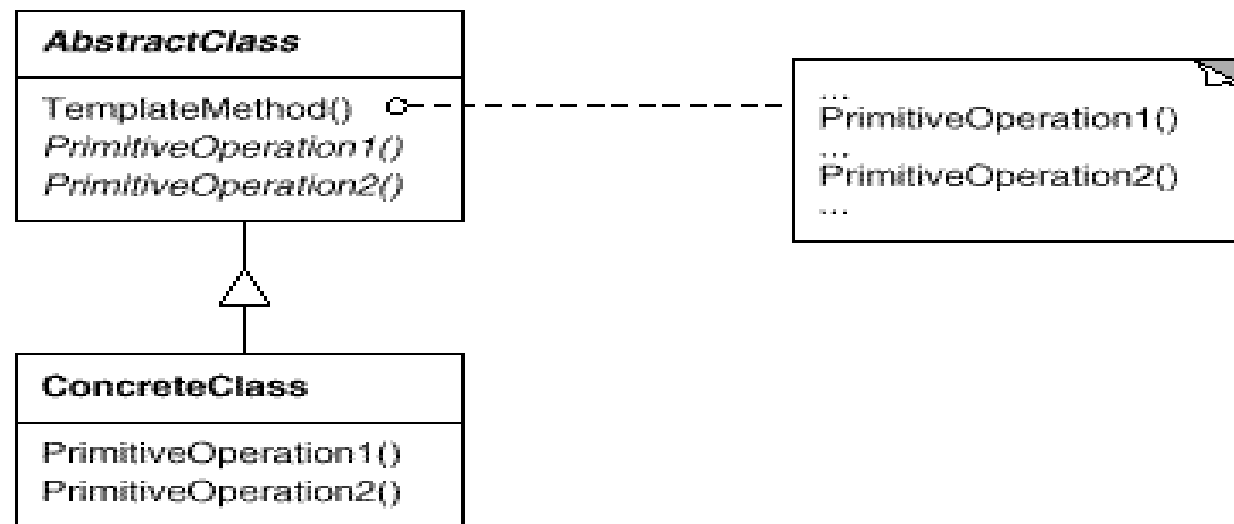


Fonte da figura: Argo Navis, curso J930

Template Method

32

□ Estrutura



PrimitiveOperation1 e **PrimitiveOperation2** são “hook methods”

Template Method – exemplo 1

33

```
public abstract class Template {  
    public abstract String link(String texto, String url);  
    public String transform(String texto) { return texto; }  
    public String templateMethod() {  
        String msg = "Endereço: " + link("Empresa", "http://www.empresa.com");  
        return transform(msg);  
    }  
}
```

```
public class XMLData extends Template {  
    public String link(String texto, String url) {  
        return "<endereco xlink:href='" + url + "'>" + texto + "</endereco>";  
    }  
}
```

```
public class HTMLData extends Template {  
    public String link(String texto, String url) {  
        return "<a href='" + url + "'>" + texto + "</a>";  
    }  
    public String transform(String texto) {  
        return texto.toLowerCase();  
    }  
}
```

Template Method – exemplo 2

34

```
3 public abstract class X {
4     public void templateMethod() {
5         System.out.println("Parte invariável 1");
6         operacaoGancho01();
7         System.out.println("Parte invariável 2");
8         operacaoGancho02();
9     }
10
11     public abstract void operacaoGancho01();
12
13     public void operacaoGancho02() {
14         System.out.println("X.operacaoGancho02()");
15     }
16 }
```

```
3 public class Y extends X {
4
5     @Override
6     public void operacaoGancho01() {
7         System.out.println("Y.operacaoGancho01()");
8     }
9
10    public void operacaoGancho02() {
11        System.out.println("Y.operacaoGancho02()");
12    }
13 }
```

```
3 public class Main {
4     public static void main(String[] args) {
5         X objeto = new Y();
6         objeto.templateMethod();
7     }
8
9 }
```

Strategy

Strategy

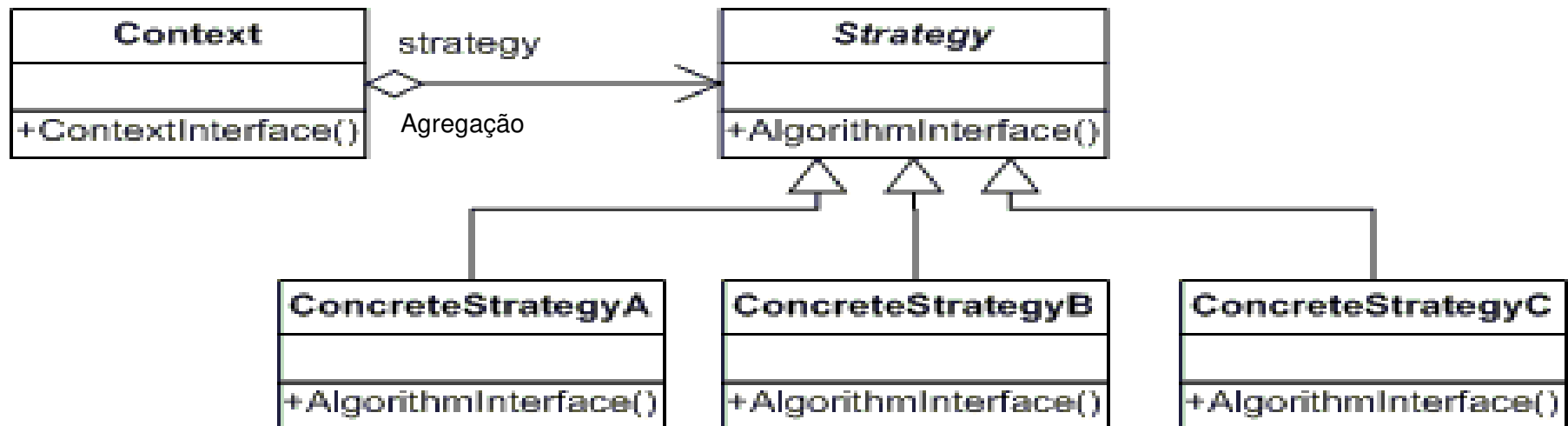
36

- Intenção: Definir uma **família de algoritmos**, **encapsular** cada um, e fazê-los **intercambiáveis**. Esse padrão permite que o algoritmo **varie independentemente** dos clientes que o utilizam.
- ▣ GoF Book: *“Define a family of algorithms, encapsulate each one, and make them interchangeable. Strategy lets the algorithm vary independently from clients that use it.”*

Strategy

37

- Estrutura



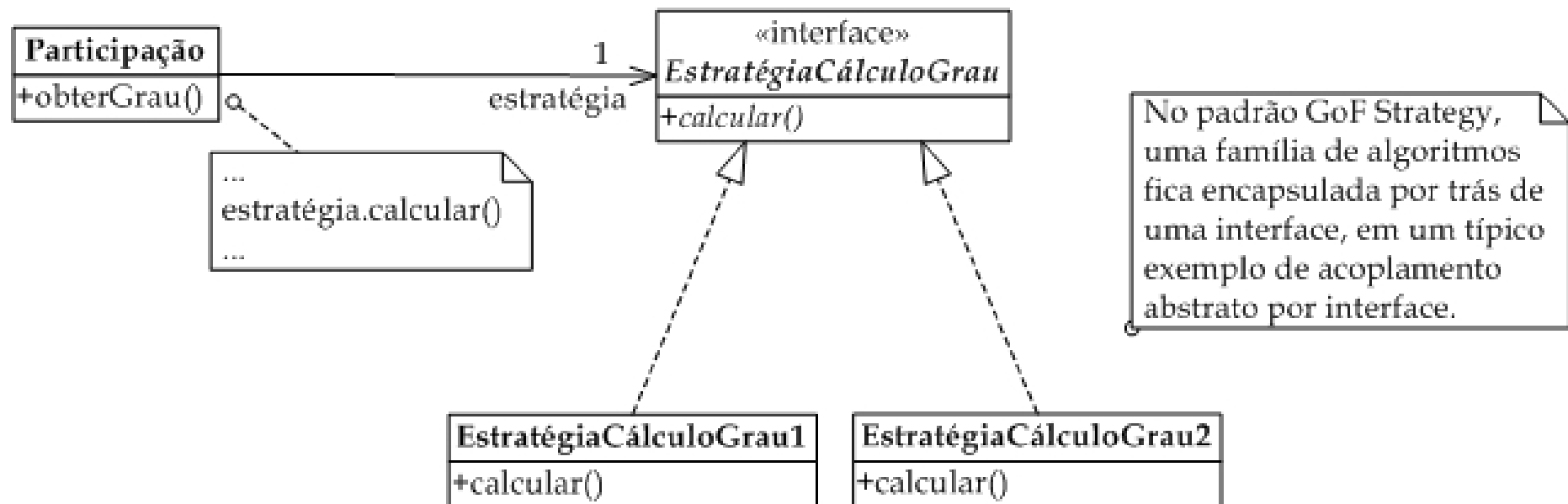
Strategy – exemplo (1 / 3)

38

- Uma instituição de ensino deve calcular o grau final cada aluno de uma turma.
 - ▣ Esse grau é uma letra (A, B, C, D ou E) e é calculado a partir de notas atribuídas a avaliações.
- O projetista identificou que, é comum a coordenação modificar a estratégia (algoritmo) de atribuição de graus a partir das notas das avaliações.
- Já existe uma classe (Participacao) que representa participações de alunos em uma turma.
- Como projetar a solução com o padrão Strategy?

Strategy – exemplo (2/3)

39



- Participacao representa participações de alunos em uma turma e suas correspondentes avaliações em termos de notas. A operação calcular retorna o grau do aluno.

Strategy – exemplo (3/3)

40

- Vantagem dessa solução: **acoplamento fraco** é obtido entre os elementos envolvidos (cliente e fornecedor).
- O cliente
 - ▣ não precisa ser alterado quando a forma de cálculo de grau tiver que ser alterada.
 - ▣ sabe apenas que existe uma operação chamada **calcular**, que realiza o cálculo.
 - ▣ não conhece a classe específica que fornece a implementação dessa operação.

```

5 public class Participacao {
6     double nota1;
7     double nota2;
8     int frequencia;
9     Turma turma;
10    int grade;
11
12+ public String obterGrau() {
17 }

```

```

12+ public String obterGrau() {
13     String grau = "";
14     double media;
15     if (grade == 2002) {
16         media = (nota1 + nota2) / 2;
17         if (media >= 9.0) {
18             grau = "A";
19         } else if (media >= 7.0) {
20             grau = "B";
21         } else if (media >= 5.0) {
22             grau = "C";
23         } else if (media >= 3.0) {
24             grau = "D";
25         } else {
26             grau = "E";
27         }
28     } else if (grade == 2007) {
29         media = nota1 * 0.6 + nota2 * 0.4;
30         if (frequencia < 0.75) {
31             grau = "E";
32         } else {
33             if (media >= 8.0) {
34                 grau = "A";
35             } else if (media >= 6.0) {
36                 grau = "B";
37             } else if (media >= 4.0) {
38                 grau = "C";
39             } else if (media >= 2.0) {
40                 grau = "D";
41             } else {
42                 grau = "E";
43             }
44         }
45     }
46     return grau;
47 }

```

Implementação
sem Strategy

```
3 public class Participacao {
4     double nota1;
5     double nota2;
6     double frequencia;
7     Turma turma;
8     int grade;
9     private StrategyCalculoGrau s;
10
11 public Participacao(StrategyCalculoGrau s) {
12     this.s = s;
13 }
14
15 public String obterGrau() {
16     return s.calcular(this);
17 }
18
19 public double getNota1() {
20     return nota1;
21 }
22
23 public double getNota2() {
24     return nota2;
25 }
26
27 public double getFrequencia() {
28     return frequencia;
29 }
30 }
31
```

```
4 public class StrategyCalculoGrauGradeNova implements StrategyCalculoGrau {
5
6     @Override
7     public String calcular(Participacao p) {
8         String grau = "";
9         double media = p.getNota1() * 0.6 + p.getNota2() * 0.4;
10        if (p.getFrequencia() < 0.75) {
11            grau = "E";
12        } else {
13            if (media >= 8.0) {
14                grau = "A";
15            } else if (media >= 6.0) {
16                grau = "B";
17            } else if (media >= 4.0) {
18                grau = "C";
19            } else if (media >= 2.0) {
20                grau = "D";
21            } else {
22                grau = "E";
23            }
24            return grau;
25        }
26    }
27 }
```

```
4 public interface StrategyCalculoGrau {
5     String calcular(Participacao p);
6 }
```

```
4 public class StrategyCalculoGauGradeAntiga implements StrategyCalculoGau {  
5  
6     @Override  
7     public String calcular(Participacao p) {  
8         String grau = "";  
9         double media = (p.getNota1() + p.getNota2()) / 2;  
10        if (media >= 9.0) {  
11            grau = "A";  
12        } else if (media >= 7.0) {  
13            grau = "B";  
14        } else if (media >= 5.0) {  
15            grau = "C";  
16        } else if (media >= 3.0) {  
17            grau = "D";  
18        } else {  
19            grau = "E";  
20        }  
21        return grau;  
22    }  
23 }
```

```
4 public interface StrategyCalculoGau {  
5     String calcular(Participacao p);  
6 }
```

ITERATOR

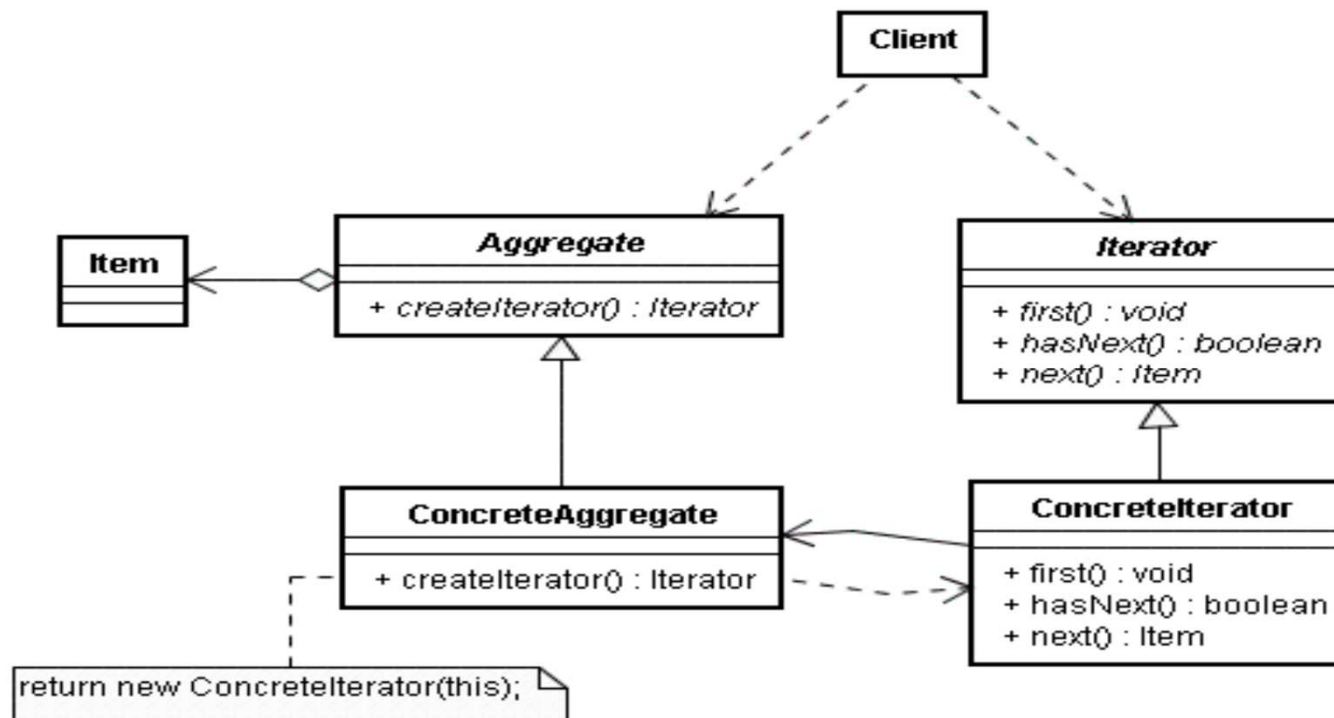
Iterator

46

- **Intenção:** percorrer seqüencialmente os itens de um **objeto agregado**, sem expor sua representação interna aos clientes.
 - ▣ GoF Book: *“Provide a way to access the elements of an aggregate object sequentially without exposing its underlying representation.”*
- Iterator permite percorrer os elementos de uma coleção sem violar o **encapsulamento** dessa coleção.

Iterator - estrutura

47



```
public class DinerMenu {
    static final int MAX_ITEMS = 6;
    int numberOfItems = 0;
    MenuItem[] menuItems;
    int position = 0;

    public DinerMenu() {
        menuItems = new MenuItem[MAX_ITEMS];
        // ....
        addItem("BLT", "Bacon with lettuce & tomato on whole wheat", false, 2.99);
        // ....
    }

    public void addItem(String name, String description, boolean vegetarian,
        double price) {
        // ....
    }

    public MenuItem[] getMenuItems() {
        return menuItems;
    }

    public Object next() {
        MenuItem menuItem = menuItems[position];
        position = position + 1;
        return menuItem;
    }

    public boolean hasNext() {
        if (position >= menuItems.length || menuItems[position] == null) {
            return false;
        } else {
            return true;
        }
    }
}
```

```
public class DinerMenuIterator implements Iterator {
    MenuItem[] items;
    int position = 0;

    public DinerMenuIterator(MenuItem[] items) {
        this.items = items;
    }

    public Object next() {
        MenuItem menuItem = items[position];
        position = position + 1;
        return menuItem;
    }

    public boolean hasNext() {
        if (position >= items.length || items[position] == null) {
            return false;
        } else {
            return true;
        }
    }
}
```

```
public class DinerMenu {
    static final int MAX_ITEMS = 6;
    int numberOfItems = 0;
    MenuItem[] menuItems;
    int position = 0;

    public DinerMenu() {
        menuItems = new MenuItem[MAX_ITEMS];
        addItem("BLT", "Bacon & Ovos", false, 2.99);
        // ....
    }

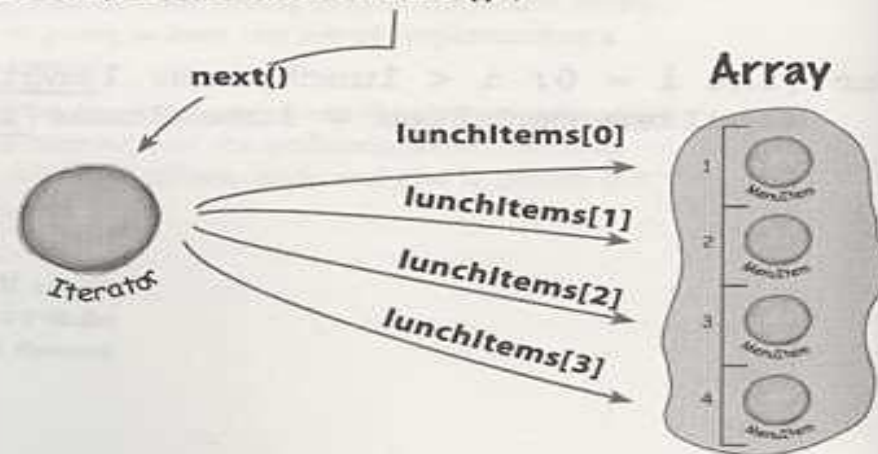
    public void addItem(String name, String description, boolean vegetarian,
        double price) {
        // ....
    }

    public Iterator<MenuItem> createIterator() {
        return new DinerMenuIterator(menuItems);
    }
}
```

```
Iterator iterator = lunchMenu.createIterator();  
  
while (iterator.hasNext()) {  
    MenuItem menuItem = (MenuItem) iterator.next();  
}
```

Wow, this code
is exactly the
same as the
breakfast/Menu
code.

Same situation here: the client just calls
hasNext() and next(); behind the scenes,
the iterator indexes into the Array.



Iterator – exemplo 2

52

- Em Java, **List**, **Set** e **SortedSet** são subinterfaces de **Collection** e herdam uma operação de assinatura **Iterator iterator()** que deve retornar um objeto **Iterator**.
 - ▣ Todas as classes que realizam essas interfaces (e.g., **ArrayList**, **Vector**, **HashSet**, **TreeSet**) devem implementar essa operação.
 - ▣ O objeto **Iterator**, por sua vez, possui as operações **hasNext()** e **next()**, que formam a interface que clientes usam para percorrer a coleção.

Referências

53

- Professor Prof. Eduardo Bezerra.
- Gamma et al. ***Padrões de Projeto - Soluções Reutilizáveis de Software Orientado a Objetos***, Porto Alegre: Bookman, 2000.
- Steven Metsker, ***Padrões de Projeto em Java***, Porto Alegre: Bookman, 2004.
- Freeman et al. ***Use a Cabeça!: Padrões de Projetos***, Rio de Janeiro: Alta Books, 2005.

Referências

54

